E-HealthCare Management System — Full Project Documentation

Ranmdy Healthcare Center

Course: Software Engineering **Technology Stack:** JavaFX 21.0.7 + JDBC + MySQL (XAMPP) + Maven **Repository:** github.com/ranmdy/hospital-management-system **Application Window:** 900 x 600 pixels

Table of Contents

1. Project Overview

Ranmdy Healthcare Center is a virtual hospital management application with three user roles:

- **Patient** — Registers, describes symptoms, gets auto-classified and matched to a doctor, chats in real time, receives prescriptions, and can be admitted or transferred.
- **Doctor** — Manages a patient queue, conducts live consultations via chat, prescribes medication, admits patients to beds, and transfers patients to partner hospitals.
- **Hospital Admin** — Receives incoming transfer requests, accepts or declines them, requests patient files, and confirms patient arrival.

The application uses JavaFX for the GUI with FXML layout files, CSS for styling, JDBC for MySQL database access through XAMPP, and Maven for dependency management and building.

---

2. Getting Started — Cloning and Setup

Step 1: Clone the Repository

```
git clone https://github.com/ranmdy/hospital-management-system.git
cd hospital-management-system
```

Step 2: Open in IntelliJ IDEA

Open IntelliJ IDEA, click “Open”, and select the cloned folder. IntelliJ will detect the `pom.xml` file and import it as a Maven project. Wait for Maven to download all dependencies.

Step 3: Install XAMPP

Download and install XAMPP from apachefriends.org. This provides Apache and MySQL (MariaDB) bundled together.

Step 4: Start MySQL in XAMPP

Open the XAMPP Control Panel and click “Start” next to MySQL. The port should be 3306 (default). This starts the MariaDB server that our application connects to.

Step 5: Verify Java and Maven

Ensure Java 21+ and Maven are installed:

```
java -version
mvn -version
```

Step 6: Build and Run

```
mvn clean compile
mvn javafx:run
```

The application window opens at 900x600 showing the login screen titled “Ranmdy Healthcare Center”.

---

3. Database Setup on XAMPP

Step 1: Open phpMyAdmin

Go to `http://localhost/phpmyadmin` in your browser.

Step 2: Create the Database

Click “New” in the left sidebar. Enter the database name: `ehealthcare`. Click “Create”.

Step 3: Run the Schema

Click on the `ehealthcare` database, then click the “SQL” tab. Paste and run the following SQL to create all 7 tables:

```sql
CREATE TABLE doctors (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    specialty VARCHAR(100),
    license_number VARCHAR(50),
    status VARCHAR(20) DEFAULT 'available'
);

CREATE TABLE patients (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    symptoms TEXT,
    illness_class VARCHAR(50),
    status VARCHAR(20) DEFAULT 'new',
    assigned_doctor_id INT,
    FOREIGN KEY (assigned_doctor_id) REFERENCES doctors(id)
);

CREATE TABLE hospitals (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    location VARCHAR(100),
    total_beds INT DEFAULT 0,
    available_beds INT DEFAULT 0
);

CREATE TABLE hospital_admins (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    hospital_id INT,
    FOREIGN KEY (hospital_id) REFERENCES hospitals(id)
);

CREATE TABLE prescriptions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    medicine VARCHAR(200) NOT NULL,
    dosage VARCHAR(200) NOT NULL,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (patient_id) REFERENCES patients(id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(id)
);

CREATE TABLE messages (
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    sender_id INT NOT NULL,
    receiver_id INT NOT NULL,
    sender_role VARCHAR(20) NOT NULL,
    content TEXT NOT NULL,
    sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE transfers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    from_hospital_id INT DEFAULT 0,
    to_hospital_id INT NOT NULL,
    urgency VARCHAR(20) DEFAULT 'routine',
    clinical_note TEXT,
    file_sent BOOLEAN DEFAULT FALSE,
    file_requested BOOLEAN DEFAULT FALSE,
    file_approved BOOLEAN DEFAULT FALSE,
    status VARCHAR(20) DEFAULT 'new',
    decline_reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (patient_id) REFERENCES patients(id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(id),
    FOREIGN KEY (to_hospital_id) REFERENCES hospitals(id)
);
```

Step 4: Verify Connection

The application connects to MySQL using these credentials (in `DatabaseConnection.java`):

- **URL:** `jdbc:mysql://localhost:3306/ehealthcare`
- **Username:** `root`
- **Password:** (empty — XAMPP default)

Note on Hospital Seeding

The hospitals table is automatically seeded with 5 sample hospitals when the application first loads and finds the table empty. The hospitals are:

Name	Location	Total Beds	Available
Lagoon Hospital	Victoria Island	120	18
Reddington Hospital	Ikeja	80	12
St. Nicholas Hospital	Lagos Island	60	8

Name	Location	Total Beds	Available
EKO Hospital	Surulere	100	15
First Consultant Hospital	Ikoyi	50	6

4. Database Schema (All 7 Tables)

doctors

Stores doctor accounts. Fields: `id`, `name`, `email`, `password`, `specialty`, `license_number`, `status` (available/busy/on_hold).

patients

Stores patient accounts and clinical data. Fields: `id`, `name`, `email`, `password`, `symptoms`, `illness_class`, `status` (new/pending/in_consult/prescribed/admitted/discharged), `assigned_doctor_id`.

hospitals

Stores partner hospitals for transfer. Fields: `id`, `name`, `location`, `total_beds`, `available_beds`.

hospital_admins

Stores hospital admin accounts linked to a hospital. Fields: `id`, `name`, `email`, `password`, `hospital_id`.

prescriptions

Stores prescriptions issued by doctors. Fields: `id`, `patient_id`, `doctor_id`, `medicine`, `dosage`, `notes`, `created_at`.

messages

Stores chat messages between doctor and patient. Fields: `id`, `sender_id`, `receiver_id`, `sender_role` (doctor/patient), `content`, `sent_at`.

transfers

Stores transfer requests from doctor to hospital. Fields: `id`, `patient_id`, `doctor_id`, `from_hospital_id`, `to_hospital_id`, `urgency` (routine/urgent/emergency), `clinical_note`, `file_sent`, `file_requested`, `file_approved`, `status` (new/accepted/declined/arrived), `decline_reason`, `created_at`.

---

5. Project Structure

```
e-hospital/
├── pom.xml                              -- Maven build configuration
├── ROADMAP.md                           -- Project development plan
└── src/
    └── main/
        ├── java/
        │   ├── module-info.java         -- Java module declaration
        │   └── com/example/ehospital/
        │       ├── HelloApplication.java          -- App entry point
        │       ├── HelloController.java           -- Unused starter controller
        │       ├── DatabaseConnection.java        -- MySQL connection helper
        │       ├── SessionManager.java            -- Tracks logged-in user
        │       ├── IllnessClassifier.java         -- Symptom classification engine
        │       ├── Patient.java                   -- Patient model
        │       ├── Doctor.java                    -- Doctor model
        │       ├── Hospital.java                  -- Hospital model
        │       ├── HospitalAdmin.java             -- Hospital admin model
        │       ├── Message.java                   -- Chat message model
        │       ├── Prescription.java              -- Prescription model
        │       ├── Transfer.java                  -- Transfer request model
        │       ├── PatientDAO.java                -- Patient database operations
        │       ├── DoctorDAO.java                 -- Doctor database operations
        │       ├── HospitalDAO.java               -- Hospital database operations
        │       ├── HospitalAdminDAO.java          -- Admin database operations
        │       ├── ChatDAO.java                   -- Message database operations
        │       ├── PrescriptionDAO.java           -- Prescription database operatio
        │       ├── TransferDAO.java               -- Transfer database operations
        │       ├── LoginController.java           -- Login screen logic
        │       ├── SignupController.java          -- Signup screen logic
        │       ├── SymptomController.java         -- Symptom intake logic
        │       ├── PatientDashboardController.java   -- Patient home screen
        │       ├── PatientChatController.java        -- Patient chat screen
        │       ├── PatientPrescriptionsController.java -- Patient Rx view
        │       ├── DoctorDashboardController.java    -- Doctor home screen
        │       ├── DoctorChatController.java         -- Doctor chat screen
        │       ├── DoctorAdmissionsController.java   -- Bed assignment screen
        │       ├── DoctorTransferController.java     -- Transfer form screen
```

```
│       ├── DoctorPrescriptionsController.java -- Doctor Rx history
│       ├── AdminDashboardController.java     -- Admin transfer inbox
│       └── AdminRecordsController.java       -- Admin patient records
└── resources/com/example/ehospital/
    ├── styles.css                    -- Global stylesheet
    ├── login-view.fxml               -- Login screen layout
    ├── signup-view.fxml              -- Signup screen layout
    ├── symptom-view.fxml             -- Symptom intake layout
    ├── patient-dashboard.fxml        -- Patient dashboard layout
    ├── patient-chat.fxml             -- Patient chat layout
    ├── patient-prescriptions.fxml    -- Patient Rx layout
    ├── doctor-dashboard.fxml         -- Doctor dashboard layout
    ├── doctor-chat.fxml              -- Doctor chat layout
    ├── doctor-admissions.fxml        -- Bed grid layout
    ├── doctor-transfer.fxml          -- Transfer form layout
    ├── doctor-prescriptions.fxml     -- Doctor Rx history layout
    ├── admin-dashboard.fxml          -- Admin inbox layout
    ├── admin-records.fxml            -- Admin records layout
    └── hello-view.fxml               -- Unused starter layout
```

6. Build Configuration

pom.xml

The Maven project uses:

- **Group ID:** `com.example`
- **Artifact ID:** `e-hospital`
- **Java source/target:** 25
- **JavaFX version:** 21.0.7

Key dependencies:

Dependency	Version	Purpose
javafx-controls	21.0.7	JavaFX UI controls (Button, Label, TextField)
javafx-fxml	21.0.7	FXML loading support
javafx-web	21.0.7	WebView component
mysql-connector-j	8.3.0	MySQL JDBC driver

Dependency	Version	Purpose
controlsfx	11.2.1	Extra JavaFX controls
formsfx-core	11.6.0	Form building library
validatorfx	0.5.0	Input validation
bootstrapfx-core	0.4.0	Bootstrap-style CSS for JavaFX
tilesfx	21.0.3	Dashboard tile components
fxgl	17.3	Game library (came with template)

The `javafx-maven-plugin` (version 0.0.8) provides the `mvn javafx:run` command which launches the application with the main class `com.example.ehospital.HelloApplication`.

module-info.java

```
module com.example.ehospital {
    requires javafx.controls;
    requires javafx.fxml;
    requires javafx.web;
    requires java.sql;
    requires org.controlsfx.controls;
    requires com.dlsc.formsfx;
    requires net.synedra.validatorfx;
    requires org.kordamp.ikonli.javafx;
    requires org.kordamp.bootstrapfx.core;
    requires eu.hansolo.tilesfx;
    requires com.almasb.fxgl.all;

    opens com.example.ehospital to javafx.fxml;
    exports com.example.ehospital;
}
```

The `opens` directive allows JavaFX to access controller classes via reflection when loading FXML files. The `exports` directive makes the package available to other modules. The `requires java.sql` is essential for JDBC database access.

---

7. Application Entry Point

HelloApplication.java

This is the main class that starts the JavaFX application.

```
public class HelloApplication extends Application {
    @Override
    public void start(Stage stage) throws Exception {
        FXMLLoader fxmlLoader = new FXMLLoader(
            HelloApplication.class.getResource("login-
        view.fxml"));
        Scene scene = new Scene(fxmlLoader.load(), 900, 600);
        stage.setTitle("Ranmdy Healthcare Center");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

What it does: Loads `login-view.fxml` as the first screen, sets the window to 900x600 pixels, and sets the title bar to “Ranmdy Healthcare Center”. The `launch()` method starts the JavaFX runtime.

How it affects the project: This is the single entry point. Every user starts at the login screen. From there, the application navigates between screens by loading different FXML files into the same Stage window.

---

8. All Java Files — Detailed Breakdown

8.1 DatabaseConnection.java

Purpose: Provides a single static method that creates a new MySQL connection.

```
public class DatabaseConnection {
    public static Connection getConnection() throws
        SQLException {
```

```
            String url = "jdbc:mysql://localhost:3306/
            ehealthcare";
            String user = "root";
            String password = "";
            return DriverManager.getConnection(url, user,
            password);
        }
    }
```

What it does: Every DAO class calls `DatabaseConnection.getConnection()` to get a fresh JDBC connection. The connection URL points to `localhost:3306` which is where XAMPP runs MySQL. The username is `root` with an empty password (XAMPP default).

How it affects the project: If XAMPP is not running or the database does not exist, every database operation will fail with a connection error. This is the single point of configuration for database access.

8.2 SessionManager.java

Purpose: Tracks the currently logged-in user across all screens using static fields.

```
public class SessionManager {
    private static Patient loggedInPatient;
    private static Doctor loggedInDoctor;
    private static HospitalAdmin loggedInAdmin;
    private static String role;
}
```

What it does: When a user logs in, one of `loginAsPatient()`, `loginAsDoctor()`, or `loginAsAdmin()` is called. This stores the user object and clears the other two. Every controller calls `SessionManager.getPatient()`, `getDoctor()`, or `getAdmin()` to know who is logged in. The `logout()` method clears everything.

How it affects the project: This is how user identity persists between screen navigations. Without it, navigating from dashboard to chat would lose track of who the user is. Only one role is active at a time.

8.3 IllnessClassifier.java

Purpose: Classifies patient symptoms into illness categories using keyword matching.

```
public class IllnessClassifier {
    public static String classify(String symptoms) { ... }
    public static String getSpecialty(String illnessClass)
        { ... }
}
```

What it does: The `classify()` method scans symptom text for keywords:

- **Respiratory** — cough, flu, breathing, asthma, lung, pneumonia
- **Cardiac** — chest pain, heart, palpitation, blood pressure, cardiac
- **Skin** — rash, acne, itch, skin, allergy, hives
- **General** — cold, headache, fatigue, nausea, fever, stomach, diarrhea, vomit
- **Other** — anything that does not match

The `getSpecialty()` method maps each class to a doctor specialty: Respiratory to Pulmonologist, Cardiac to Cardiologist, Skin to Dermatologist, General and Other to General.

How it affects the project: This drives the automatic doctor matching. When a patient submits symptoms, the classifier determines which specialty of doctor they need, and the system searches for an available doctor of that specialty.

8.4 Model Classes

Patient.java

Fields: `id`, `name`, `email`, `password`, `symptoms`, `illnessClass`, `status`, `assignedDoctorId`

The `status` field tracks the patient through their lifecycle: new → pending → in_consult → prescribed/admitted/discharged. The `assignedDoctorId` links to the doctor handling them.

Doctor.java

Fields: `id`, `name`, `email`, `password`, `specialty`, `licenseNumber`, `status`

The `status` field is one of: available, busy, on_hold. The `specialty` is used for doctor matching (e.g., “Pulmonologist”, “Cardiologist”).

Hospital.java

Fields: `id`, `name`, `location`, `totalBeds`, `availableBeds`

Tracks bed capacity for partner hospitals. `availableBeds` decrements when a transfer is accepted and increments when a bed is freed.

HospitalAdmin.java

Fields: `id`, `name`, `email`, `password`, `hospitalId`

Links an admin account to a specific hospital via `hospitalId`.

Message.java

Fields: `id`, `senderId`, `receiverId`, `senderRole`, `content`, `sentAt`

Each message records who sent it (doctor or patient), who received it, and the timestamp. The `senderRole` field determines which side of the chat the bubble appears on.

Prescription.java

Fields: `id`, `patientId`, `doctorId`, `medicine`, `dosage`, `notes`, `createdAt`

Stores a single prescription with the medication name, dosage instructions, optional notes, and timestamp.

Transfer.java

Fields: `id`, `patientId`, `doctorId`, `fromHospitalId`, `toHospitalId`, `urgency`, `clinicalNote`, `fileSent`, `fileRequested`, `fileApproved`, `status`, `declineReason`, `createdAt`

The most complex model. Tracks the full lifecycle of a transfer request: new → accepted/declined → arrived. The file-sharing booleans track whether the

patient file was sent immediately, requested by the hospital, or approved by the doctor.

8.5 DAO Classes (Data Access Objects)

PatientDAO.java

Methods: - `register(name, email, password)` — INSERT into patients table - `login(email, password)` — SELECT matching patient - `getById(id)` — Get single patient by ID - `saveSymptoms(patientId, symptoms, illnessClass)` — UPDATE symptoms and illness class, set status to 'pending' - `assignDoctor(patientId, doctorId)` — UPDATE assigned_doctor_id - `updateStatus(patientId, status)` — UPDATE status field - `getPendingForDoctor(doctorId)` — SELECT all pending patients assigned to a doctor - `getPendingBySpecialty(specialty)` — SELECT pending patients whose doctor has a given specialty - `getInConsultForDoctor(doctorId)` — SELECT the patient currently being consulted by a doctor - `getAdmittedForDoctor(doctorId)` — SELECT all admitted patients for a doctor (used for bed grid)

How it affects the project: This is the most used DAO. Every patient flow (registration, symptom submission, doctor assignment, status changes) goes through PatientDAO. The `getPendingForDoctor()` method drives the doctor's queue. The `getInConsultForDoctor()` method determines which patient the doctor is currently talking to.

DoctorDAO.java

Methods: - `register(name, email, password, specialty, licenseNumber)` — INSERT into doctors table - `login(email, password)` — SELECT matching doctor - `getById(id)` — Get single doctor by ID - `findAvailableBySpecialty(specialty)` — Find an available doctor: first tries exact specialty match (case-insensitive), then falls back to any available doctor - `updateStatus(doctorId, status)` — UPDATE status field

How it affects the project: The `findAvailableBySpecialty()` method is critical for doctor matching. It first tries to find a doctor with the exact specialty the patient needs, then falls back to any available doctor. This ensures patients are never stuck without a doctor if one is available.

HospitalDAO.java

Methods: - `getAll()` — SELECT all hospitals (calls `seedIfEmpty()` first) - `seedIfEmpty()` — INSERT 5 sample hospitals if table is empty - `getById(id)` — Get single hospital by ID - `getByAdminId(adminId)` — Get hospital linked to an admin via JOIN - `decrementBed(hospitalId)` — Decrease available_beds by 1 (when transfer accepted) - `incrementBed(hospitalId)` — Increase available_beds by 1 (when bed freed, capped at total_beds)

How it affects the project: The auto-seeding ensures the hospital dropdown always has options, even on first run. The bed count methods maintain accurate capacity tracking when transfers are accepted.

HospitalAdminDAO.java

Methods: - `register(name, email, password, hospitalId)` — INSERT with hospital link - `register(name, email, password)` — Overload that sets hospitalId to NULL - `login(email, password)` — SELECT matching admin

How it affects the project: The hospitalId link is essential. When an admin logs in, the system uses their hospitalId to find their hospital and load only transfers directed to that hospital.

ChatDAO.java

Methods: - `sendMessage(senderId, receiverId, senderRole, content)` — INSERT message - `getMessages(userId1, userId2)` — SELECT all messages between two users in chronological order

How it affects the project: The chat system uses a simple bidirectional query: `(sender=A AND receiver=B) OR (sender=B AND receiver=A)` ordered by timestamp. Both the patient and doctor chat screens poll this every 2 seconds to show new messages.

PrescriptionDAO.java

Methods: - `save(patientId, doctorId, medicine, dosage, notes)` — INSERT prescription - `getByPatientId(patientId)` — Get most recent prescription for a patient - `getAllByPatientId(patientId)` — Get all prescriptions for a patient - `getAllByDoctorId(doctorId)` — Get all prescriptions issued by a doctor

How it affects the project: Prescriptions link doctors to patients. The patient dashboard shows the latest prescription, the prescriptions screens show the full history.

TransferDAO.java

Methods: - `create(patientId, doctorId, toHospitalId, urgency, clinicalNote, fileSent)` — INSERT transfer - `getByHospitalId(hospitalId)` — SELECT all transfers for a hospital (admin inbox) - `getById(id)` — Get single transfer by ID - `getByDoctorAndPatient(doctorId, patientId)` — Get most recent transfer - `getActiveByDoctorAndPatient(doctorId, patientId)` — Get active transfer (status is 'new' or 'accepted') - `updateStatus(id, status)` — UPDATE status field - `decline(id, reason)` — UPDATE status to 'declined' and set decline_reason - `requestFile(id)` — SET file_requested = TRUE - `approveFile(id)` — SET file_approved = TRUE

How it affects the project: Manages the entire transfer lifecycle. The admin dashboard loads transfers via `getByHospitalId()`. The doctor chat screen checks for active transfers via `getActiveByDoctorAndPatient()` to show transfer status and file request banners.

8.6 Controller Classes

LoginController.java

Screen: login-view.fxml **FXML fields:** emailField, passwordField, patientCard, doctorCard, adminCard, messageLabel

What it does: Shows three role cards (Patient, Doctor, Hospital Admin). User clicks a card to select their role, enters email and password, and clicks "Sign in". The controller creates the appropriate DAO, calls `login()`, and if successful, stores the user in SessionManager and navigates to their dashboard.

Key behaviors: - `selectPatient()`, `selectDoctor()`, `selectAdmin()` — Toggle role card styling (active/inactive) - `onLogin()` — Validates fields, authenticates against the database, navigates to the correct dashboard - `goToSignup()` — Navigates to the signup screen

SignupController.java

Screen: signup-view.fxml **FXML fields:** nameField, emailField, passwordField, specialtyField, licenseField, hospitalBox, hospitalCombo, role cards, messageLabel

What it does: Registration form that adapts based on role. Patient just needs name/email/password. Doctor additionally needs specialty and license number. Admin additionally needs to select a hospital from the dropdown.

Key behaviors: - `initialize()` — Loads hospitals into the dropdown via HospitalDAO - Role selection toggles visibility of specialty/license fields (doctor) or hospital dropdown (admin) - `onSignup()` — Validates fields, calls the appropriate DAO register method, shows success or “email already exists” error

SymptomController.java

Screen: symptom-view.fxml **FXML fields:** symptomArea, classChip, classLabel, specialtyLabel, classHint, queueHint, submitBtn, messageLabel

What it does: Patient describes their symptoms in a text area, clicks “Classify” to get the illness category and doctor specialty, then submits to join the doctor queue.

Key behaviors: - `onClassify()` — Calls `IllnessClassifier.classify()` and `getSpecialty()`, shows results as a chip - `onSubmit()` — Saves symptoms, finds available doctor of matching specialty (with General fallback), assigns doctor, shows confirmation message, then auto-redirects to dashboard after 2 seconds using a Timer

Timer management: The redirect timer is stored as a field and cancelled when navigating away to prevent stale callbacks.

PatientDashboardController.java

Screen: patient-dashboard.fxml **FXML fields:** greetingLabel, subLabel, avatarLabel, userNameLabel, statusLabel, consultTitle, consultDesc, doctorAvatar, doctorNameLabel, doctorSpecLabel, doctorStatusLabel, rxCard, rxCardText, admitCardText, doctorLicenseLabel, consultButton

What it does: Shows the patient’s current care status — greeting with date, status pill, consultation card with illness classification, assigned doctor info with availability indicator, latest prescription card, and admission notice.

Key behaviors: - `initialize()` — Refreshes patient from DB, sets greeting with first name and today's date, updates status pill color (in_consult=red, prescribed/discharged=green, admitted=red, pending=yellow), loads doctor info, loads latest prescription - `goToConsultation()` — Refreshes patient from DB to check if doctor was assigned since page loaded, then navigates to chat or symptom screen

PatientChatController.java

Screen: patient-chat.fxml **FXML fields:** avatarLabel, userNameLabel, doctorChatAvatar, doctorChatName, doctorChatSpec, chatScroll, chatBox, messageField, rxPanel, detailsName, detailsInfo

What it does: Real-time chat with the assigned doctor. Messages appear as styled bubbles (patient messages right-aligned blue, doctor messages left-aligned white). Polls every 2 seconds for new messages. Shows prescription card when doctor issues one.

Key behaviors: - `loadMessages()` — Only reloads if message count changed (efficiency optimization) - `checkPrescription()` — Shows styled prescription card (navy header + body) or empty state (dashed border) - `startPolling()` — Timer every 2 seconds calls loadMessages() + checkPrescription() via Platform.runLater() - `stopPolling()` — Cancels timer before any navigation to prevent leaks

PatientPrescriptionsController.java

Screen: patient-prescriptions.fxml **FXML fields:** avatarLabel, userNameLabel, rxList

What it does: Shows all prescriptions issued to the patient. Each prescription is rendered as a styled card with a navy header, patient/prescriber info row, medication box (blue background), notes section, and digitally signed footer.

DoctorDashboardController.java

Screen: doctor-dashboard.fxml **FXML fields:** avatarLabel, userNameLabel, specialtyLabel, doctorStatusLabel, topSubLabel, queueTitle, emptyLabel, queueList, currentPatientCard, currentPatientAvatar, currentPatientName, currentPatientSymptoms, currentPatientClass, btnAvailable, btnBusy, btnOnHold

What it does: Shows the doctor's current status, patient queue with NEXT badge, and current patient card. Includes an availability toggle (Available / Busy / On Hold).

Key behaviors: - `updateAvailToggle()` — Highlights the active status button - `loadCurrentPatient()` — Shows info about the patient currently in consultation - `loadQueue()` — Lists pending patients with Accept button - `acceptPatient()` — Blocks if doctor already consulting someone (shows feedback), otherwise sets patient to in_consult and doctor to busy, navigates to chat - `setOnHold()` — Sets doctor to on_hold, **re-queues pending patients** to another available doctor of the same specialty - `onNextFromDash()` — Discharges current patient (only if still in_consult), sets doctor to available, refreshes

DoctorChatController.java

Screen: doctor-chat.fxml **FXML fields:** avatarLabel, userNameLabel, docSpecLabel, patientChatAvatar, patientChatName, patientChatInfo, patientStatusPill, chatScroll, chatBox, messageField, symptomText, classChipLabel, prescribeBtn, rxForm, medicineField, dosageField, notesField, rxMessage, afterConsult, transferStatusBox, transferStatusCard, transferHospitalLabel, transferStatusLabel, fileRequestBox, fileRequestLabel, approveFileBtn

What it does: The doctor's consultation screen. Chat with patient on the left, clinical tools on the right. Right panel includes: symptom summary with illness chip, clinical decision buttons (Prescribe / Admit / Transfer), prescription form (toggles on click), transfer status card (shows pending/ accepted/declined), file request banner, and after-consult options (Next patient / Go on hold).

Key behaviors: - `loadTransferStatus()` — Checks for active transfer, shows status card with color coding (yellow=pending, green=accepted, red=declined), shows file request banner if hospital requested file - `onPrescribe()` — Toggles prescription form visibility - `onIssuePrescription()` — Saves prescription, updates patient status to "prescribed", sends system message in chat, shows after-consult buttons - `onAdmit()` — Sets patient status to "admitted", sends system message - `onViewFile()` — Opens Alert dialog showing patient's full medical record (name, email, status, illness, symptoms, prescription) - `onApproveFile()` — Approves file sharing for transfer, sends system message - `onNextPatient()` — Stops polling, sets doctor available, auto-accepts first pending patient or goes to dashboard - `onHold()` — Stops polling, sets doctor on_hold, **re-queues pending patients**, goes to dashboard

DoctorAdmissionsController.java

Screen: doctor-admissions.fxml **FXML fields:** avatarLabel, userNameLabel, specialtyLabel, admitContent

What it does: Shows a 6-bed grid. Occupied beds show patient name (dimmed). Free beds show “Tap to assign” and are clickable. Bottom pairing bar shows current patient waiting for bed assignment. If all beds full, shows warning with “Prescribe instead” button.

Key behaviors: - `initialize()` — Builds the entire UI programmatically (title, bed grid, pairing bar, warning) - `assignBed(bedId)` — Sets patient to “admitted”, sends chat message with bed ID, refreshes screen - Free bed cards have click handlers: `bedCard.setOnMouseClicked(e -> assignBed(bed))` - No-beds warning includes “Prescribe instead” button that navigates back to doctor-chat.fxml

DoctorTransferController.java

Screen: doctor-transfer.fxml **FXML fields:** avatarLabel, userNameLabel, docSpecLabel, patientAvatar, patientNameLabel, patientInfoLabel, hospitalCombo, btnRoutine, btnUrgent, btnEmergency, reasonField, fileSendCard, fileRequestCard, messageLabel

What it does: Form to transfer the current patient to a partner hospital. Doctor selects destination hospital from dropdown, urgency level (Routine/Urgent/Emergency), writes clinical notes, and chooses file sharing option (send now or let hospital request).

Key behaviors: - `initialize()` — Loads current patient and hospital list into dropdown - Urgency buttons toggle with active/inactive styling - File sharing cards toggle with border color change - `onSubmit()` — Validates fields, creates transfer via TransferDAO, sends system message in chat, auto-redirects to chat after 1.5 seconds

DoctorPrescriptionsController.java

Screen: doctor-prescriptions.fxml **FXML fields:** avatarLabel, userNameLabel, specialtyLabel, rxList

What it does: Shows all prescriptions the doctor has issued. Each card shows patient info, medication, dosage, and date. Empty state shows dashed border with message.

AdminDashboardController.java

Screen: admin-dashboard.fxml **FXML fields:** avatarLabel, userNameLabel, hospitalInfoLabel, totalBedsLabel, occupiedLabel, freeBedsLabel, inboxTitle, inboxList, detailPanel, detailAvatar, detailPatientName, detailPatientInfo, detailUrgencyPill, detailStatusPill, detailDoctorName, detailDate, detailReason, fileStatusLabel, fileActionBtn, actionButtons, resultBox, resultLabel, arrivedBtn

What it does: Hospital admin's main screen. Top bar shows hospital stats (capacity, occupied, free beds). Left column shows transfer request inbox. Right column shows selected request detail with urgency pill, status pill, requesting doctor, date, clinical note, file status, and action buttons.

Key behaviors: - `loadTransfers()` — Loads all transfers for this hospital, builds inbox rows with avatars, names, urgency/status pills, and NEW badges - `selectTransfer(t)` — Populates detail panel with full transfer info, shows appropriate buttons/status - `onAccept()` — Updates transfer to "accepted", decrements bed count, marks patient as "admitted", refreshes - `onDecline()` — Shows TextInputDialog for decline reason, saves decline - `onMarkArrived()` — Updates transfer to "arrived", refreshes (button only visible for accepted transfers) - `onRequestFile()` — Marks transfer as file_requested

AdminRecordsController.java

Screen: admin-records.fxml **FXML fields:** avatarLabel, userNameLabel, topSubLabel, listTitle, patientList, detailPanel, detailAvatar, detailName, detailEmail, detailStatusPill, detailClass, detailDoctor, detailTransferStatus, detailSymptoms, detailClinicalNote, detailRxList, detailChatList

What it does: Shows all patients who were transferred to this hospital. Left column lists unique patients with status pills. Right column shows full clinical record: demographics, illness classification, assigned doctor, transfer status, symptoms, clinical notes, all prescriptions, and full chat transcript.

Key behaviors: - Uses a HashSet to show unique patients (avoids duplicates from multiple transfers) - Shows color-coded status pills (admitted=red, prescribed=green, other=gray) - Loads full chat transcript between doctor and patient

---

9. All FXML Files — Detailed Breakdown

Every screen uses the same layout pattern: a navy sidebar on the left with navigation buttons, and a main content area on the right with a cream (#F4F1EC) background.

login-view.fxml

Three role cards (Patient/Doctor/Admin) with radio-style selection, email and password fields, Sign In button, and link to signup. Uses `LoginController`.

signup-view.fxml

Name, email, password fields. Specialty and license fields (visible for doctor). Hospital dropdown (visible for admin). Role cards same as login. Uses `SignupController`.

symptom-view.fxml

Large text area for symptom description, “Classify symptoms” button, classification result chip (hidden until classified), queue hint label, “Submit & start consultation” button. Uses `SymptomController`.

patient-dashboard.fxml

Greeting card with date, status pill, consultation card (illness class + specialty), doctor info card (avatar, name, specialty, status, license), prescription card, admission card. Uses `PatientDashboardController`.

patient-chat.fxml

Split view — chat area (left) with message bubbles and input field, side panel (right) with doctor info, “Your Details” section, and prescription panel. Polls every 2 seconds. Uses `PatientChatController`.

patient-prescriptions.fxml

List of prescription cards with navy header, patient/prescriber info, medication box, notes, and signed footer. Uses `PatientPrescriptionsController`.

doctor-dashboard.fxml

Two-column layout. Left: availability toggle (Available/Busy/On Hold), current patient card. Right: patient queue with NEXT badge, Accept buttons, status lifecycle legend. Uses `DoctorDashboardController`.

doctor-chat.fxml

Split view — chat area (left) with message bubbles and input, right panel with: symptom summary + classification chip, "View patient file" button, clinical decision buttons (Prescribe/Admit/Transfer), prescription form (hidden), transfer status card (hidden), file request banner (hidden), after-consult options (hidden). Uses `DoctorChatController`.

doctor-admissions.fxml

Simple layout with sidebar and content VBox. The bed grid, pairing bar, and warnings are all built programmatically in the controller. Uses `DoctorAdmissionsController`.

doctor-transfer.fxml

Transfer form with: patient info bar, hospital dropdown, urgency toggle (Routine/Urgent/Emergency), clinical notes text area, file sharing option cards (Send now / Let hospital request). Uses `DoctorTransferController`.

doctor-prescriptions.fxml

List of prescription cards showing patient avatar, name, illness class, medication, dosage, notes, and date. Uses `DoctorPrescriptionsController`.

admin-dashboard.fxml

Hospital stats bar (capacity/occupied/free), two-column layout: transfer inbox (left) with clickable rows, detail panel (right) with urgency/status pills, doctor/date info, clinical note, file status with request button, accept/decline buttons, result box with "Mark as arrived" button. Uses `AdminDashboardController`.

admin-records.fxml

Two-column layout: patient list (left) with status pills, detail panel (right) with demographics, classification, doctor, transfer status, symptoms, clinical notes, prescriptions list, and chat transcript. Uses `AdminRecordsController`.

---

10. Stylesheet (styles.css)

The application uses a cohesive design system:

Colors: - Navy: #13294C (sidebar, headers) - Cream: #F4F1EC (main background) - Blue: #1F4D8F (primary buttons, links) - Teal: #127566 (available status, success) - Purple: #4B3FA6 (accents, arrived status) - Red: #B14A33 (error, emergency, busy) - Yellow: #7A5A12 (urgent, pending)

Key CSS classes: - `.sidebar` — Navy background, fixed width - `.nav-item` / `.nav-item-active` — Sidebar navigation buttons - `.card` — White rounded card with border - `.btn-primary` — Blue action button - `.btn-logout` — Transparent logout button - `.text-input` — Styled text field with rounded border - `.status-available` — Green pill - `.status-busy` — Red pill - `.status-pending` — Yellow pill - `.role-card` / `.role-card-active` — Login/signup role selection cards - `.avatar` / `.avatar-blue` — Circular avatar labels with initials - `.error-label` — Red error text - `.success-label` — Green success text

---

11. Development Timeline — Commit by Commit

Commit 1: `44dbef7` — Initial commit

Empty repository created on GitHub.

Commit 2: `451544e` — Revise README

Updated project title and description for the E-Healthcare Management system.

Commit 3: `1a12244` — Add project roadmap

Created ROADMAP.md documenting the full development plan with 10 chunks (0-9).

Commit 4: `1bfec12` — Update print statement

Small test change — updated “Hello” to “Goodbye” to verify the repo works.

Commit 5: `5a2c264` — Add JavaFX project scaffold

Set up the base JavaFX 17 project with Maven, module-info, starter application class, and .gitignore.

Commit 6: `179941d` — Add files via upload

Additional project files uploaded.

Commit 7: `fdc96cd` — Chunk 0 and Chunk 1: Database + Login/Signup

This was the first major commit. Created the database schema with all 7 tables. Added MySQL JDBC connector. Built the three-role authentication system: - Patient, Doctor, HospitalAdmin models and DAOs - LoginController with role card selection - SignupController with conditional field visibility - SessionManager for session tracking - Blank dashboards for all three roles - Upgraded JavaFX from 17 to 21.0.7 for macOS compatibility

Commit 8: `c213e39` — Chunks 2-3: Symptoms + Doctor Queue

Added the symptom intake system and doctor matching: - IllnessClassifier with keyword-based classification - SymptomController with classify-then-submit flow - Doctor queue with pending patient list - Accept flow (doctor goes busy, patient goes in_consult) - Doctor specialty matching with General fallback - Styled UI with navy sidebar and cream background

Commit 9: `c31fcb7` — New fixes

Bug fixes and minor improvements.

Commit 10: `12939a6` — Match all screens to design reference

Major UI overhaul to match the frontend design: - Patient dashboard: added date subtitle, doctor license, prescription card, Open consultation button - Symptom view: restructured with auto-classify result box, pill chips, queue hint - Doctor dashboard: two-column layout, availability toggle, NEXT badge, status legend - Patient chat: online indicator, Your Details section - Doctor chat: Transfer to hospital button - Auth screens: updated footer text

Commit 11: `ac02099` — Transfer system + Hospital admin dashboard

Added the entire transfer feature: - Transfer model and DAO - Doctor transfer form with hospital dropdown, urgency toggle, clinical notes, file sharing - Hospital model and DAO with auto-seeding - Admin dashboard with stats bar, transfer inbox, detail panel - Accept/decline/file request flow - Hospital admin signup with hospital selection

Commit 12: `6a7477c` — Fix transfer flow + Patient records + Bug fixes

Comprehensive bug fix commit: - Transfer accept now decrements beds and admits patient - Decline shows reason dialog - Doctor chat shows transfer status and file request banner - New admin patient records screen - Fixed admission beds not clickable - Fixed polling timer leaks - Fixed null safety across 10 controllers - Fixed NPE-unsafe .equals() patterns - Auto-seed hospitals table

Commit 13: `3b7c1ac` — Remaining roadmap features

Final feature commit: - Re-queue pending patients when doctor goes on hold - “Mark as arrived” button for accepted transfers - incrementBed() in HospitalDAO - “Prescribe instead” button when beds full - “View patient file” dialog in doctor chat

---

12. Bugs Found and Fixed

Bug 1: Empty Hospital Dropdown

File: `HospitalDAO.java` **Problem:** The hospitals table was empty in the database. When the doctor tried to create a transfer, the hospital dropdown was empty with nothing to select. **Fix:** Added `seedIfEmpty()` method that checks `SELECT COUNT(*)` and inserts 5 sample hospitals if the count is zero. Called automatically before `getAll()`.

Bug 2: Transfer Accept Did Not Decrement Beds

File: `AdminDashboardController.java` **Problem:** When the admin accepted a transfer, the status updated but the hospital's available bed count did not decrease. The stats bar showed incorrect numbers. **Fix:** Added `decrementBed()` call in `onAccept()` and also added `patientDAO.updateStatus()` to mark the patient as "admitted".

Bug 3: Transfer Decline Had No Reason

File: `AdminDashboardController.java` **Problem:** The decline button updated the status to "declined" but there was no way for the admin to specify a reason. The doctor just saw "Declined" with no explanation. **Fix:** Added a `TextInputDialog` in `onDecline()` that prompts for the decline reason. The reason is saved via `transferDAO.decline()` and displayed in the doctor's transfer status card.

Bug 4: Doctor Could Not See Transfer Status

File: `DoctorChatController.java`, `doctor-chat.fxml` **Problem:** After submitting a transfer, the doctor had no visibility into whether it was accepted, declined, or pending. They had to guess. **Fix:** Added `loadTransferStatus()` method that checks for active transfers via `getActiveByDoctorAndPatient()`. Shows a styled card with color-coded status (yellow=pending, green=accepted, red=declined with reason). Polled every 2 seconds alongside messages.

Bug 5: Doctor Could Not See File Requests

File: `DoctorChatController.java`, `doctor-chat.fxml` **Problem:** When the hospital admin requested the patient file, the doctor had no notification and

no way to approve the sharing. **Fix:** Added file request banner in the chat sidebar that appears when `transfer.isFileRequested()` is true and file is not yet approved. Added `onApproveFile()` method that calls `transferDAO.approveFile()`.

Bug 6: Admission Beds Not Clickable

File: `DoctorAdmissionsController.java` **Problem:** The free bed cards showed "Tap to assign" text but had no click handler. Clicking them did nothing. **Fix:** Added `bedCard.setOnMouseClicked(e -> assignBed(bed))` on free bed cards. The `assignBed()` method sets the patient to "admitted", sends a chat message with the bed ID, and refreshes the screen.

Bug 7: Login Error Not Styled

File: `LoginController.java` **Problem:** Error messages appeared as plain text without the red error styling. The `messageLabel.setText()` was called without setting the CSS class first. **Fix:** Added `messageLabel.getStyleClass().setAll("error-label")` before all error messages (4 places in the login flow).

Bug 8: Accept Patient Silent Failure

File: `DoctorDashboardController.java` **Problem:** When a doctor was already consulting a patient and clicked "Accept" on another pending patient, nothing happened. No feedback was given. **Fix:** Added a check for existing in-consult patient. If one exists, show feedback message: "Finish your current consultation first." in `topSubLabel` with red styling.

Bug 9: getInitials() NullPointerException

File: All 10 controller files **Problem:** The `getInitials()` method would crash with a NullPointerException if the name was null or empty. This could happen with newly created accounts or corrupted data. **Fix:** Added null guard at the top of every `getInitials()` method:

```
if (name == null || name.isEmpty()) return "?";
```

This was applied to: LoginController, SignupController, SymptomController, PatientDashboardController, PatientChatController, PatientPrescriptionsController, DoctorDashboardController, DoctorChatController, DoctorAdmissionsController, DoctorTransferController,

DoctorPrescriptionsController, AdminDashboardController, AdminRecordsController.

Bug 10: Polling Timer Leaks

Files: `PatientChatController.java`, `DoctorChatController.java`
Problem: Polling timers started even when there was no patient or doctor loaded. This wasted resources and could cause NullPointerExceptions when the timer callback tried to access null objects. **Fix:** Added null guards before starting polling. In PatientChatController: only start if `doctor != null`. In DoctorChatController: only start if `patient != null`.

Bug 11: Unmanaged Redirect Timers

Files: `SymptomController.java`, `DoctorTransferController.java`
Problem: The redirect timers (used to auto-navigate after 2 seconds) were created as anonymous objects. If the user navigated away before the timer fired, the callback would try to load a screen on a disposed stage. **Fix:** Stored timer references as instance fields (`redirectTimer`). Added cancellation in `loadScreen()` before loading the new scene.

Bug 12: NPE-Unsafe .equals() Calls

Files: `AdminDashboardController.java`, `DoctorDashboardController.java`, `PatientDashboardController.java`
Problem: Code like `status.equals("value")` crashes if `status` is null. This happened with newly registered patients/doctors whose status field had not been set yet. **Fix:** Flipped all comparisons to the NPE-safe pattern: `"value".equals(status)`. This way, if status is null, the comparison returns false instead of throwing an exception.

Bug 13: Patient Records Button Dead

File: `admin-dashboard.fxml`, `AdminDashboardController.java` **Problem:** The “Patient records” button in the admin sidebar had no `onAction` handler. Clicking it did nothing. **Fix:** Created `AdminRecordsController.java` and `admin-records.fxml` for the patient records screen. Wired the button with `onAction="#goToRecords"` and added the `goToRecords()` method.

Bug 14: Missing Status Pill Styles

File: `PatientDashboardController.java` **Problem:** The “admitted” and “pending” patient statuses had no visual styling in the dashboard status pill. They appeared as unstyled text. **Fix:** Added cases for “admitted” (status-busy, red) and “pending” (status-pending, yellow) in the status pill styling logic.

Bug 15: Stale Patient Data on Consultation

File: `PatientDashboardController.java` **Problem:** The “Open consultation” button checked `patient.getAssignedDoctorId()` from the session, which could be stale if a doctor was assigned after the page loaded. **Fix:** Added a fresh `PatientDAO.getById()` call in `goToConsultation()` before checking the doctor assignment.

---

13. Final Feature Summary

Feature	Status	Files Involved
Patient registration and login	Done	LoginController, SignupController, PatientDAO
Doctor registration and login	Done	LoginController, SignupController, DoctorDAO
Hospital admin registration and login	Done	LoginController, SignupController, HospitalAdminDAO
Session management (3 roles)	Done	SessionManager
Symptom intake and classification	Done	SymptomController, IllnessClassifier
Auto doctor matching by specialty	Done	SymptomController, DoctorDAO
Doctor patient queue	Done	DoctorDashboardController, PatientDAO

Feature	Status	Files Involved
Doctor accept patient (one at a time)	Done	DoctorDashboardController
Real-time chat (2s polling)	Done	DoctorChatController, PatientChatController, ChatDAO
Prescription form and issuance	Done	DoctorChatController, PrescriptionDAO
Prescription view (patient)	Done	PatientPrescriptionsController
Prescription history (doctor)	Done	DoctorPrescriptionsController
Patient dashboard with care status	Done	PatientDashboardController
Doctor dashboard with availability toggle	Done	DoctorDashboardController
Bed admission grid (6 beds)	Done	DoctorAdmissionsController
Transfer request form	Done	DoctorTransferController, TransferDAO
Admin transfer inbox	Done	AdminDashboardController
Transfer accept/ decline with reason	Done	AdminDashboardController
File request and approval flow	Done	AdminDashboardController, DoctorChatController
Doctor transfer status view	Done	DoctorChatController
Admin patient records screen	Done	AdminRecordsController
Mark as arrived button	Done	AdminDashboardController
	Done	

Feature	Status	Files Involved
Re-queue patients on doctor hold		DoctorDashboardController, DoctorChatController
Prescribe fallback when beds full	Done	DoctorAdmissionsController
Doctor patient file viewer	Done	DoctorChatController
Hospital auto-seeding	Done	HospitalDAO
Bed count management	Done	HospitalDAO (increment/ decrement)

End of Documentation